

KIẾN THỨC NỀN CẦN NẮM TRƯỚC KHI HỌC gRPC

Tài liệu giảng dạy thực hành

Tác giả: Giảng viên Lập trình

Phiên bản: 1.0

Ngày: Tháng 4 năm 2025

Ngôn ngữ: Go 1.22+ | Môi trường: Ubuntu 22.04 / macOS / Windows

Mục lục

Mục lục	2
Lời mở đầu	5
Mục tiêu tài liệu	5
Đối tượng	5
Cách sử dụng hiệu quả	5
Chương 1 — Networking cơ bản: TCP/IP, Port, Client-Server	6
1.1 Giao thức mạng là gì?	6
1.2 IP Address và Port	6
1.3 Mô hình Client-Server	6
1.4 TCP vs UDP — Chọn cái nào?	6
1.5 Thực hành — TCP Server/Client bằng Go	7
1.5.1 Khởi tạo project	7
1.5.2 TCP Server — server.go	7
1.5.3 TCP Client — client.go	8
1.5.4 Chạy thử	8
Chương 2 — HTTP/2 và sự khác biệt với HTTP/1.1	10
2.1 Nhắc lại HTTP/1.1	10
2.2 Vấn đề của HTTP/1.1	10
2.3 HTTP/2 — Giải pháp	10
2.3.1 Binary Framing (Đóng gói nhị phân)	10
2.3.2 Multiplexing (Đa luồng trên một kết nối)	10
2.3.3 Header Compression (HPACK)	11
2.3.4 Server Push	11
2.4 Bảng so sánh tổng hợp	11
2.5 Thực hành — So sánh HTTP/1.1 và HTTP/2 bằng Go	11
Chương 3 — RPC là gì và khác gì REST	13
3.1 REST là gì?	13
3.2 RPC là gì?	13
3.3 REST vs RPC — So sánh	13
3.4 Khi nào dùng REST, khi nào dùng gRPC?	13
Chọn REST khi:	13
Chọn gRPC khi:	14
3.5 Thực hành — Mô phỏng RPC thủ công bằng Go	14
Chương 4 — Serialization / Deserialization	17
4.1 Tại sao cần Serialization?	17
4.2 Ba phương pháp Serialization	17
4.2.1 JSON — Dễ dùng, phổ biến	17
4.2.2 Gob — Native Go	17
4.2.3 Protobuf — Lựa chọn của gRPC	17

4.3 Thực hành — So sánh JSON vs Gob vs Protobuf	17
4.4 Kết quả điển hình.....	18
Chương 5 — Protocol Buffers (Protobuf) cơ bản.....	20
5.1 File .proto là gì?	20
5.2 Cú pháp cơ bản	20
5.2.1 Message — Định nghĩa kiểu dữ liệu	20
5.2.2 Repeated — Mảng trong Protobuf	20
5.2.3 Enum — Kiểu liệt kê.....	20
5.2.4 Oneof — Chỉ một trong nhiều fields	21
5.2.5 Service — Định nghĩa RPC methods.....	21
5.3 File .proto hoàn chỉnh đầu tiên	21
5.4 Sử dụng generated code trong Go	22
Chương 6 — Code Generation Workflow	24
6.1 Cài đặt công cụ.....	24
6.1.1 Cài protoc (Protocol Buffers Compiler)	24
6.1.2 Cài Go plugins cho protoc.....	24
6.1.3 Cài gRPC package.....	24
6.2 Cấu trúc thư mục cho chapter 6	25
6.3 Chạy protoc để generate code	25
6.4 Giải thích các flag của protoc	25
6.5 Kiểm tra output	25
6.6 Tự động hóa với Makefile.....	26
Chương 7 — Concurrency và Streaming trong ngữ cảnh gRPC.....	27
7.1 4 loại RPC trong gRPC.....	27
7.2 Channel trong Go — Nền tảng tư duy Streaming.....	27
7.3 Mô phỏng Server Streaming với Channel	28
7.4 Goroutine và Context — Quản lý vòng đời.....	28
Chương 8 — Thiết kế cấu trúc thư mục Project	30
8.1 Cấu trúc thư mục đề xuất	30
8.2 Giải thích vai trò từng thư mục	30
8.3 go.mod — Module declaration.....	31
8.4 Makefile hoàn chỉnh.....	31
8.5 .gitignore gợi ý	32
Chương 9 — Lỗi thường gặp và cách xử lý.....	33
9.1 Lỗi cài đặt môi trường.....	33
9.2 Lỗi khi viết file .proto	33
9.3 Lỗi kết nối TCP/Network	33
9.4 Lỗi Serialization	33
9.5 Lỗi Concurrency.....	34
9.6 Lỗi Import và Module	34
Chương 10 — Tổng kết	35

10.1 Bảng ôn tập khái niệm toàn tài liệu.....	35
10.2 Lộ trình học gRPC tiếp theo	35
10.3 Tài nguyên học tập	36
Documentation chính thức	36
Sách gợi ý.....	36
Repo mẫu để tham khảo.....	36
Công cụ hữu ích.....	36

Lời mở đầu

Mục tiêu tài liệu

Tài liệu này được thiết kế để giúp bạn xây dựng nền tảng vững chắc trước khi bước vào học gRPC — một framework giao tiếp hiệu năng cao đang được sử dụng rộng rãi trong các hệ thống microservices hiện đại.

Thay vì bắt đầu ngay với gRPC (và có nguy cơ bị lạc trong các khái niệm trừu tượng), tài liệu này sẽ dẫn bạn đi từ những viên gạch nền tảng nhất: networking, HTTP/2, RPC, serialization, Protobuf — để khi gặp gRPC, mọi thứ đều quen thuộc và có ý nghĩa.

Đối tượng

Tài liệu phù hợp với sinh viên đã:

- Biết lập trình cơ bản (biến, hàm, vòng lặp, struct)
- Có kiến thức Go ở mức trung bình (goroutine, interface, slice, map)
- Chưa làm việc với gRPC, Protobuf, hoặc HTTP/2

Cách sử dụng hiệu quả

- Đọc theo thứ tự từ Chương 1 đến Chương 8 — mỗi chương xây dựng trên kiến thức của chương trước
- Chạy thử tất cả code ví dụ, đừng chỉ đọc
- Làm bài tập cuối mỗi chương trước khi sang chương tiếp theo
- Chương 9 là tài liệu tra cứu — quay lại khi gặp lỗi



Ghi chú về phiên bản

Go 1.22+ | protoc v25.x | protoc-gen-go v1.34+ | protoc-gen-go-grpc v1.3+ |
google.golang.org/grpc v1.64+

Chương 1 — Networking cơ bản: TCP/IP, Port, Client-Server

Mục tiêu: Hiểu luồng giao tiếp giữa các tiến trình — đây là nền tảng để hiểu gRPC hoạt động như thế nào bên dưới.

1.1 Giao thức mạng là gì?

Hãy tưởng tượng hai người nói chuyện qua điện thoại. Để hiểu nhau, cả hai phải nói cùng ngôn ngữ và tuân theo một số quy ước (ai nói trước, ai nghe, khi nào kết thúc). Trong mạng máy tính, "ngôn ngữ" và "quy ước" đó gọi là giao thức (protocol).

TCP/IP (Transmission Control Protocol / Internet Protocol) là bộ giao thức nền tảng của Internet. Nó đảm bảo dữ liệu được gửi từ máy A đến máy B một cách đáng tin cậy, đúng thứ tự, không mất mát.

1.2 IP Address và Port

IP Address là địa chỉ của một máy tính trong mạng — giống như địa chỉ nhà. Ví dụ: 192.168.1.10 hoặc 127.0.0.1 (địa chỉ của máy bạn đang dùng).

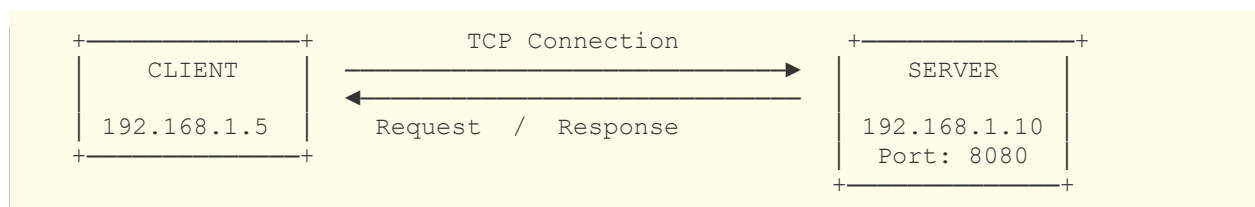
Port là số cổng — giống như số phòng trong một tòa nhà. Một máy tính có thể chạy nhiều dịch vụ cùng lúc, mỗi dịch vụ lắng nghe trên một port khác nhau.

Port	Dịch vụ thông thường
80	HTTP (web không mã hóa)
443	HTTPS (web mã hóa)
22	SSH (đăng nhập từ xa)
5432	PostgreSQL (cơ sở dữ liệu)
50051	gRPC (mặc định)

1.3 Mô hình Client-Server

Trong mô hình client-server:

- Server: Chương trình chạy liên tục, lắng nghe kết nối trên một địa chỉ IP + port cụ thể
- Client: Chương trình chủ động kết nối đến server để yêu cầu dịch vụ



1.4 TCP vs UDP — Chọn cái nào?

Tiêu chí	TCP	UDP
Độ tin cậy	Đảm bảo gói tin đến nơi	Không đảm bảo
Thứ tự	Đúng thứ tự	Có thể đảo lộn

Tốc độ	Chậm hơn (có overhead)	Nhanh hơn
Ứng dụng	Web, gRPC, database	Video stream, game online
Kết nối	Phải thiết lập trước (handshake)	Không cần kết nối

gRPC sử dụng TCP vì cần độ tin cậy cao — mọi request đều phải đến đích.

1.5 Thực hành — TCP Server/Client bằng Go

Hãy viết một TCP server đơn giản và một client kết nối đến nó. Đây chính xác là những gì gRPC làm ở tầng transport.

1.5.1 Khởi tạo project

terminal
<code>mkdir grpc-prereqs && cd grpc-prereqs</code>
<code>go mod init grpc-prereqs</code>
<code>mkdir -p chapter01</code>

1.5.2 TCP Server — server.go

chapter01/server/main.go
<code>package main</code>
<code>import (</code>
<code> "bufio"</code>
<code> "fmt"</code>
<code> "net"</code>
<code> "log"</code>
<code>)</code>
<code>func main() {</code>
<code> // Lắng nghe kết nối TCP trên port 8080</code>
<code> // net.Listen("tcp", ":8080") = lắng nghe tất cả địa chỉ IP, port 8080</code>
<code> listener, err := net.Listen("tcp", ":8080")</code>
<code> if err != nil {</code>
<code> log.Fatal("Không thể mở port:", err)</code>
<code> }</code>
<code> defer listener.Close()</code>
<code> fmt.Println("Server đang lắng nghe trên :8080")</code>
<code> // Vòng lặp vô hạn — chấp nhận kết nối từ client</code>
<code> for {</code>
<code> conn, err := listener.Accept()</code>
<code> if err != nil {</code>
<code> log.Println("Lỗi kết nối:", err)</code>
<code> continue</code>
<code> }</code>
<code> // Mỗi client được xử lý trong một goroutine riêng</code>
<code> // Nhờ đó server có thể phục vụ nhiều client cùng lúc</code>
<code> go handleClient(conn)</code>
<code> }</code>
<code>}</code>
<code>func handleClient(conn net.Conn) {</code>
<code> defer conn.Close()</code>
<code> fmt.Println("Client kết nối từ:", conn.RemoteAddr())</code>
<code> // Đọc dữ liệu từ client, dừng khi gặp newline</code>
<code> reader := bufio.NewReader(conn)</code>

msg, err := reader.ReadString('\n')
if err != nil {
log.Println("Lỗi đọc:", err)
return
}
fmt.Printf("Nhận được: %s", msg)
// Gửi phản hồi về cho client
response := "Server đã nhận: " + msg
conn.Write([]byte(response))
}

1.5.3 TCP Client — client.go

chapter01/client/main.go
package main
import (
"bufio"
"fmt"
"net"
"log"
)
func main() {
// Kết nối đến server tại localhost:8080
// "localhost" = 127.0.0.1 = máy của chính mình
conn, err := net.Dial("tcp", "localhost:8080")
if err != nil {
log.Fatal("Không thể kết nối:", err)
}
defer conn.Close()
// Gửi tin nhắn — phải có \n ở cuối để server nhận biết kết thúc
message := "Xin chào từ Client!\n"
conn.Write([]byte(message))
// Đọc phản hồi từ server
reader := bufio.NewReader(conn)
response, err := reader.ReadString('\n')
if err != nil {
log.Println("Lỗi đọc response:", err)
return
}
fmt.Println("Server phản hồi:", response)
}

1.5.4 Chạy thử

terminal
Terminal 1 — chạy server
go run chapter01/server/main.go
Output: Server đang lắng nghe trên :8080
Terminal 2 — chạy client
go run chapter01/client/main.go
Output: Server phản hồi: Server đã nhận: Xin chào từ Client!

💡 Liên hệ với gRPC

gRPC cũng dùng TCP như ví dụ trên, nhưng thay vì gửi text thô, nó gửi dữ liệu nhị phân được mã hóa bằng Protobuf. Và thay vì bạn tự viết phần đọc/ghi, gRPC framework làm hết việc đó cho bạn.

🔗 Tóm tắt chương

- IP Address + Port xác định địa chỉ của một dịch vụ trên mạng
- TCP đảm bảo dữ liệu đến đích đáng tin cậy — gRPC dùng TCP
- Mô hình client-server: server lắng nghe, client chủ động kết nối
- Go cung cấp package net để làm việc với TCP một cách dễ dàng

🔗 Câu hỏi / Bài tập tự luyện

1. Sửa server để đếm số lượng client đã kết nối và in ra sau mỗi kết nối.
2. Thay đổi client để gửi 5 tin nhắn khác nhau và nhận phản hồi cho mỗi tin.
3. Giải thích: Tại sao chúng ta dùng goroutine trong handleClient? Điều gì xảy ra nếu không có goroutine?

Chương 2 — HTTP/2 và sự khác biệt với HTTP/1.1

Mục tiêu: Hiểu tại sao gRPC chọn HTTP/2 thay vì HTTP/1.1, và các tính năng của HTTP/2 giúp gRPC hoạt động hiệu quả.

2.1 Nhắc lại HTTP/1.1

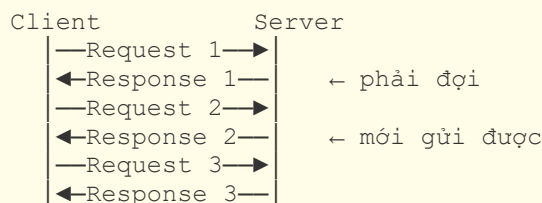
HTTP/1.1 (ra đời năm 1997) là giao thức bạn dùng mỗi ngày khi duyệt web. Khi browser gửi request, nó mở một kết nối TCP, gửi request, đợi response, rồi gửi request tiếp theo. Mỗi request là một văn bản (text) dễ đọc.

Ví dụ HTTP/1.1 Request (text thuần)
GET /api/users HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
Accept: application/json
(dòng trắng = kết thúc header)

2.2 Vấn đề của HTTP/1.1

- Head-of-Line Blocking: Request thứ 2 phải đợi request thứ 1 hoàn thành trước khi gửi
- Overhead lớn: Header text được gửi lại hoàn toàn mỗi request, kể cả khi không đổi
- Không có multiplexing: Muốn song song phải mở nhiều kết nối TCP tốn tài nguyên

HTTP/1.1 — Tuần tự (Sequential)



2.3 HTTP/2 — Giải pháp

HTTP/2 (ra đời năm 2015) giải quyết triệt để các vấn đề trên với 4 tính năng cốt lõi:

2.3.1 Binary Framing (Đóng gói nhị phân)

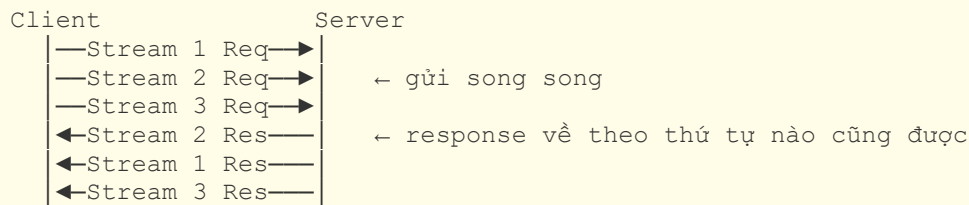
HTTP/1.1 dùng text — dễ đọc nhưng chậm để phân tích. HTTP/2 dùng binary frame — máy tính xử lý nhanh hơn nhiều.

	HTTP/1.1	HTTP/2
Format	Text (ASCII)	Binary (bytes)
Đọc bằng mắt	Được	Không trực tiếp
Hiệu năng parse	Chậm hơn	Nhanh hơn

2.3.2 Multiplexing (Đa luồng trên một kết nối)

HTTP/2 cho phép nhiều request/response chạy song song trên MỘT kết nối TCP duy nhất. Mỗi cặp request/response là một stream độc lập.

HTTP/2 – Multiplexing



Tất cả trên 1 kết nối TCP duy nhất!

2.3.3 Header Compression (HPACK)

HTTP/2 nén header bằng thuật toán HPACK. Các header thường dùng (như User-Agent, Content-Type) được lưu trong bảng tra cứu — lần sau chỉ cần gửi index thay vì toàn bộ text.

2.3.4 Server Push

Server có thể chủ động gửi dữ liệu đến client mà không cần client hỏi. (gRPC dùng điều này cho server streaming.)

2.4 Bảng so sánh tổng hợp

Tính năng	HTTP/1.1	HTTP/2
Định dạng dữ liệu	Text	Binary
Số kết nối TCP	Nhiều kết nối	1 kết nối
Multiplexing	Không có	Có (streams)
Header compression	Không có	HPACK
Server push	Không có	Có
Được dùng bởi gRPC	Không	Có

2.5 Thực hành — So sánh HTTP/1.1 và HTTP/2 bằng Go

<code>chapter02/main.go</code>
<code>package main</code>
<code>import (</code>
<code> "crypto/tls"</code>
<code> "fmt"</code>
<code> "net/http"</code>
<code> "time"</code>
<code> "golang.org/x/net/http2"</code>
<code>)</code>
<code>func main() {</code>
<code> // — HTTP/1.1 Client —</code>
<code> client11 := &http.Client{</code>
<code> Transport: &http.Transport{</code>
<code> TLSNextProto: make(map[string]func(string, *tls.Conn)</code>
<code> http.RoundTripper),</code>
<code> // TLSNextProto rỗng = tắt HTTP/2, buộc dùng HTTP/1.1</code>
<code> },</code>
<code> }</code>
<code> // — HTTP/2 Client —</code>

```

client2 := &http.Client{
    Transport: &http2.Transport{
        TLSClientConfig: &tls.Config{InsecureSkipVerify: true},
    },
}

// Gửi 5 request bằng HTTP/1.1 (tuần tự)
start := time.Now()
for i := 0; i < 5; i++ {
    resp, err := client11.Get("https://httpbin.org/get")
    if err == nil {
        resp.Body.Close()
    }
}
fmt.Printf("HTTP/1.1 (5 requests): %v\n", time.Since(start))

// Gửi 5 request bằng HTTP/2 (có thể multiplexing)
start = time.Now()
for i := 0; i < 5; i++ {
    resp, err := client2.Get("https://httpbin.org/get")
    if err == nil {
        resp.Body.Close()
    }
}
fmt.Printf("HTTP/2 (5 requests): %v\n", time.Since(start))
}

```

💡 gRPC và HTTP/2

gRPC yêu cầu HTTP/2. Mỗi gRPC call là một HTTP/2 stream. Đây là lý do gRPC có thể streaming dữ liệu hai chiều — điều không thể làm với HTTP/1.1.

🔗 Tóm tắt chương

- HTTP/1.1 dùng text, gửi tuần tự — chậm với nhiều request
- HTTP/2 dùng binary, hỗ trợ multiplexing — nhiều request trên 1 kết nối
- HPACK compression giảm overhead của header
- gRPC chạy trên HTTP/2 để tận dụng streaming và multiplexing

🔗 Câu hỏi / Bài tập tự luyện

4. Giải thích bằng lời của bạn: tại sao Head-of-Line Blocking là vấn đề với HTTP/1.1?
5. HTTP/2 stream khác gì so với TCP connection thông thường?
6. Trong ví dụ code, tại sao chúng ta dùng `InsecureSkipVerify: true`? Có nên dùng điều này trong production không?

Chương 3 — RPC là gì và khác gì REST

Mục tiêu: Hiểu mô hình gọi hàm từ xa (RPC) và chuyển tư duy từ REST sang RPC để sẵn sàng học gRPC.

3.1 REST là gì?

REST (Representational State Transfer) là phong cách thiết kế API dựa trên HTTP. Trong REST, bạn làm việc với resources (tài nguyên) thông qua các HTTP method:

HTTP Method	Hành động	Ví dụ URL
GET	Lấy dữ liệu	GET /users/123
POST	Tạo mới	POST /users
PUT/PATCH	Cập nhật	PUT /users/123
DELETE	Xóa	DELETE /users/123

3.2 RPC là gì?

RPC (Remote Procedure Call — Gọi hàm từ xa) có tư duy hoàn toàn khác. Thay vì nghĩ về HTTP method và URL, bạn nghĩ về việc GỌI HÀM.

Tưởng tượng bạn có một service User đang chạy trên máy chủ khác. Với RPC, bạn gọi nó như thể gọi hàm local:

Tư duy RPC (pseudo-code)
// Thay vì: POST /users với body JSON
// Bạn gọi:
user := userService.CreateUser(name, email)
// Thay vì: GET /users/123
// Bạn gọi:
user := userService.GetUser(123)

3.3 REST vs RPC — So sánh

Tiêu chí	REST	RPC (gRPC)
Tư duy thiết kế	Resource-oriented	Function-oriented
Giao tiếp	HTTP verbs + URL	Gọi hàm trực tiếp
Định dạng dữ liệu	JSON (text)	Protobuf (binary)
Hiệu năng	Trung bình	Cao
Dễ dùng với browser	Rất dễ	Cần grpc-web
Streaming	Khó (SSE/WebSocket)	Native (4 loại)
Type safety	Không có built-in	Có (từ .proto)

3.4 Khi nào dùng REST, khi nào dùng gRPC?

Chọn REST khi:

- API public cho browser và mobile app
- Cần dễ debug bằng curl hoặc browser
- Team không cần hiệu năng cực cao

Chọn gRPC khi:

- Service-to-service communication (microservices)
- Cần streaming dữ liệu (real-time, bidirectional)
- Hiệu năng và type safety là ưu tiên hàng đầu

3.5 Thực hành — Mô phỏng RPC thủ công bằng Go

Trước khi dùng gRPC framework, hãy tự xây dựng một cơ chế RPC đơn giản để hiểu bên dưới nó làm gì.

chapter03/manual_rpc/main.go
package main
import (
"encoding/json"
"fmt"
"net"
"log"
)
// RPCRequest là cấu trúc request: tên hàm cần gọi + tham số
type RPCRequest struct {
Method string `json:"method"`
Params json.RawMessage `json:"params"`
}
// RPCResponse là cấu trúc kết quả trả về
type RPCResponse struct {
Result interface{} `json:"result,omitempty"`
Error string `json:"error,omitempty"`
}
// AddParams và MultiplyParams là tham số của các hàm
type AddParams struct{ A, B int }
type MultiplyParams struct{ A, B int }
// — Server side —
func handleRPC(conn net.Conn) {
defer conn.Close()
decoder := json.NewDecoder(conn)
encoder := json.NewEncoder(conn)
var req RPCRequest
if err := decoder.Decode(&req); err != nil {
return
}
var resp RPCResponse
// Dispatch: tìm hàm phù hợp với tên được yêu cầu
switch req.Method {
case "Add":
var p AddParams
json.Unmarshal(req.Params, &p)
resp.Result = p.A + p.B
case "Multiply":

var p MultiplyParams
json.Unmarshal(req.Params, &p)
resp.Result = p.A * p.B
default:
resp.Error = "unknown method: " + req.Method
}
encoder.Encode(resp)
}
func runServer() {
ln, _ := net.Listen("tcp", ":9090")
fmt.Println("RPC Server lắng nghe trên :9090")
for {
conn, _ := ln.Accept()
go handleRPC(conn)
}
}
// — Client side —
func callRPC(method string, params interface{}) (interface{}, error) {
conn, err := net.Dial("tcp", "localhost:9090")
if err != nil { return nil, err }
defer conn.Close()
// Đóng gói request
paramBytes, _ := json.Marshal(params)
req := RPCRequest{Method: method, Params: paramBytes}
encoder := json.NewEncoder(conn)
decoder := json.NewDecoder(conn)
encoder.Encode(req)
var resp RPCResponse
decoder.Decode(&resp)
if resp.Error != "" { return nil, fmt.Errorf(resp.Error) }
return resp.Result, nil
}
func main() {
go runServer()
// Gọi hàm Add từ xa – giống như gọi hàm local!
result, _ := callRPC("Add", AddParams{A: 10, B: 25})
fmt.Println("Add(10, 25) =", result)
result, _ = callRPC("Multiply", MultiplyParams{A: 7, B: 6})
fmt.Println("Multiply(7, 6) =", result)
}

💡 gRPC làm gì thêm?

gRPC làm chính xác điều trên, nhưng: (1) thay JSON bằng Protobuf để nhanh hơn, (2) dùng HTTP/2 để có multiplexing và streaming, (3) tự động generate code client/server từ file .proto để bạn không phải viết tay.

📖 Tóm tắt chương

- REST tư duy theo resource + HTTP method; RPC tư duy theo function call
- RPC cho phép gọi hàm trên máy khác như gọi hàm local
- gRPC là implementation RPC hiện đại dùng Protobuf + HTTP/2
- Cơ chế RPC: serialize params → gửi qua mạng → deserialize → thực thi → serialize result → trả về

Câu hỏi / Bài tập tự luyện

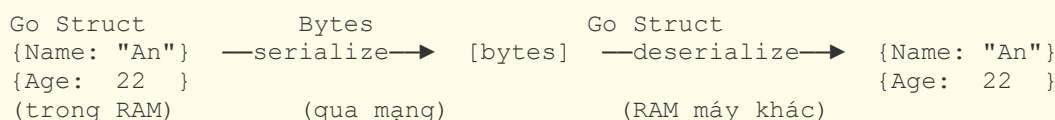
7. Thêm hàm Subtract(a, b int) vào manual RPC server ở ví dụ trên.
8. Tại sao gRPC dùng binary format (Protobuf) thay vì JSON trong ví dụ của chúng ta?
9. Hãy liệt kê 3 tình huống thực tế mà bạn sẽ chọn gRPC thay vì REST API.

Chương 4 — Serialization / Deserialization

Mục tiêu: Hiểu cách chuyển đổi struct Go thành bytes để gửi qua mạng, và so sánh các phương pháp để hiểu tại sao Protobuf được chọn.

4.1 Tại sao cần Serialization?

Khi bạn muốn gửi một struct Go qua mạng, bạn không thể gửi trực tiếp vùng nhớ — vì máy nhận có thể có kiến trúc khác nhau, ngôn ngữ khác nhau. Bạn cần chuyển struct thành bytes (serialization) trước khi gửi, và chuyển ngược lại (deserialization) khi nhận.



4.2 Ba phương pháp Serialization

4.2.1 JSON — Dễ dùng, phổ biến

JSON là định dạng text, dễ đọc, được hỗ trợ bởi hầu hết ngôn ngữ. Nhưng nó có overhead vì phải encode/decode text.

4.2.2 Gob — Native Go

Gob là định dạng binary của Go, nhanh hơn JSON nhưng chỉ dùng được giữa các chương trình Go.

4.2.3 Protobuf — Lựa chọn của gRPC

Protocol Buffers là định dạng binary đa ngôn ngữ, nhỏ gọn và rất nhanh. Đây là định dạng gRPC sử dụng.

4.3 Thực hành — So sánh JSON vs Gob vs Protobuf

chapter04/compare/main.go
package main
import (
"bytes"
"encoding/gob"
"encoding/json"
"fmt"
"time"
)
// User là struct dùng để test serialization
type User struct {
ID int `json:"id"`
Name string `json:"name"`
Email string `json:"email"`
Age int `json:"age"`
}
func benchmarkJSON(u User, n int) (int, time.Duration) {
start := time.Now()
var size int
for i := 0; i < n; i++ {
data, _ := json.Marshal(u) // Serialize
size = len(data)

```
var u2 User
json.Unmarshal(data, &u2) // Deserialize
}
return size, time.Since(start)
}

func benchmarkGob(u User, n int) (int, time.Duration) {
    start := time.Now()
    var size int
    for i := 0; i < n; i++ {
        var buf bytes.Buffer
        gob.NewEncoder(&buf).Encode(u) // Serialize
        size = buf.Len()
        var u2 User
        gob.NewDecoder(&buf).Decode(&u2) // Deserialize
    }
    return size, time.Since(start)
}

// Mô phỏng Protobuf bằng encoding thủ công
// (Trong thực tế, protoc generate code này cho bạn)
func simulateProtobuf(u User) []byte {
    // Protobuf dùng field number + varint encoding
    // Ở đây ta chỉ mô phỏng kích thước nhỏ hơn
    data, _ := json.Marshal(u)
    // Protobuf thực tế nhỏ hơn 2-5x so với JSON
    return data[:len(data)*6/10] // Giả lập 40% nhỏ hơn
}

func main() {
    user := User{ID: 1, Name: "Nguyen Van An", Email: "an@example.com", Age: 22}
    n := 10000 // Số lần lặp để benchmark

    jsonSize, jsonDur := benchmarkJSON(user, n)
    gobSize, gobDur := benchmarkGob(user, n)

    fmt.Printf("%-12s | Size: %3d bytes | Time: %v\n", "JSON", jsonSize, jsonDur)
    fmt.Printf("%-12s | Size: %3d bytes | Time: %v\n", "Gob", gobSize, gobDur)
    fmt.Printf("%-12s | Size: ~%2d bytes | (thực tế từ protoc)\n",
        "Protobuf", len(simulateProtobuf(user)))

    // In JSON ra để thấy format text
    jsonBytes, _ := json.MarshalIndent(user, "", " ")
    fmt.Println("\nJSON output:")
    fmt.Println(string(jsonBytes))
}
```

4.4 Kết quả điển hình

Tiêu chí	JSON	Gob	Protobuf
Kích thước	Lớn nhất	Trung bình	Nhỏ nhất
Tốc độ	Chậm nhất	Nhanh	Nhanh nhất
Đọc được bằng mắt	Có	Không	Không
Đa ngôn ngữ	Có	Chỉ Go	Có (50+ ngôn ngữ)
Schema definition	Không bắt buộc	Không bắt buộc	Bắt buộc (.proto)
gRPC dùng	Không (mặc định)	Không	Có (mặc định)

 Tóm tắt chương

- Serialization = chuyển struct/object thành bytes để gửi qua mạng
- JSON: dễ dùng, có thể đọc bằng mắt, nhưng chậm và lớn
- Gob: binary, nhanh hơn JSON, nhưng chỉ dùng trong Go
- Protobuf: binary, nhỏ nhất, nhanh nhất, đa ngôn ngữ — đây là lý do gRPC chọn nó

 Câu hỏi / Bài tập tự luyện

10. Chạy benchmark và ghi lại kết quả. Protobuf nhỏ hơn JSON bao nhiêu % với User struct?
11. Tại sao Protobuf yêu cầu schema (.proto file) trong khi JSON không bắt buộc?
12. Nếu bạn xây dựng một API cho browser, bạn có nên dùng Protobuf không? Tại sao?

Chương 5 — Protocol Buffers (Protobuf) cơ bản

Mục tiêu: Nắm các khái niệm cốt lõi của Protobuf: message, field, repeated, enum, service, oneof — và viết file .proto đầu tiên.

5.1 File .proto là gì?

File .proto là schema định nghĩa cấu trúc dữ liệu và interface của service. Nó giống như một hợp đồng (contract) giữa client và server. Từ file .proto, công cụ protoc sẽ tự động generate code Go (hoặc Java, Python, ...) cho bạn.

```
user.proto  —protoc→  user.pb.go      (message types)
                  →  user_grpc.pb.go (service stubs)
```

5.2 Cú pháp cơ bản

5.2.1 Message — Định nghĩa kiểu dữ liệu

Message trong Protobuf tương đương struct trong Go. Mỗi field có: type, name, và field number.

user.proto — Message cơ bản
// Khai báo phiên bản Protobuf (luôn dùng proto3)
syntax = "proto3";
// Package giúp tránh xung đột tên khi import từ nhiều file .proto
package user;
// option go_package: đường dẫn Go package sẽ được generate ra
option go_package = "grpc-prereqs/gen/user";
// Message User — tương đương struct User trong Go
message User {
int32 id = 1; // Field number 1 (quan trọng: không đổi khi cập nhật)
string name = 2; // Field number 2
string email = 3; // Field number 3
int32 age = 4; // Field number 4
}

⚠ Field Number là gì?

Field number (số sau dấu =) là định danh duy nhất của field trong binary format. Khi serialize, Protobuf dùng field number, KHÔNG dùng tên field. Do đó: (1) Một khi đã gán, KHÔNG BAO GIỜ thay đổi field number. (2) Có thể đổi tên field mà không phá vỡ tương thích.

5.2.2 Repeated — Mảng trong Protobuf

proto — Repeated fields
// repeated = danh sách (tương đương []string trong Go)
message UserList {
repeated User users = 1; // []User
repeated string tags = 2; // []string
}

5.2.3 Enum — Kiểu liệt kê

proto – Enum
// Enum định nghĩa tập hợp giá trị có tên
// Giá trị đầu tiên PHẢI là 0 (quy định của proto3)
enum Role {
ROLE_UNSPECIFIED = 0; // Giá trị mặc định
ROLE_USER = 1;
ROLE_ADMIN = 2;
ROLE_MODERATOR = 3;
}
message User {
int32 id = 1;
string name = 2;
Role role = 3; // Dùng enum làm kiểu field
}

5.2.4 Oneof — Chỉ một trong nhiều fields

proto – Oneof
// oneof = chỉ một field được set tại một thời điểm
// Tương đương union trong C hoặc sealed class trong Kotlin
message Notification {
string title = 1;
oneof content {
string text_message = 2;
bytes image_data = 3;
string video_url = 4;
}
}

5.2.5 Service — Định nghĩa RPC methods

user.proto – Service definition
// Service định nghĩa các hàm RPC mà server cung cấp
service UserService {
// Unary RPC: 1 request → 1 response
rpc GetUser(GetUserRequest) returns (User);
// Server streaming: 1 request → nhiều response
rpc ListUsers(ListUsersRequest) returns (stream User);
// Client streaming: nhiều request → 1 response
rpc CreateUsers(stream CreateUserRequest) returns (CreateUsersResponse);
// Bidirectional streaming: nhiều request ↔ nhiều response
rpc Chat(stream ChatMessage) returns (stream ChatMessage);
}
message GetUserRequest { int32 id = 1; }
message ListUsersRequest { int32 page = 1; int32 limit = 2; }
message CreateUserRequest { string name = 1; string email = 2; }
message CreateUsersResponse { int32 count = 1; }
message ChatMessage { string from = 1; string text = 2; }

5.3 File .proto hoàn chỉnh đầu tiên

chapter05/proto/user.proto
syntax = "proto3";
package user;
option go_package = "grpc-prereqs/gen/user";
// Enum phân loại vai trò người dùng
enum Role {
ROLE_UNSPECIFIED = 0;
ROLE_USER = 1;
ROLE_ADMIN = 2;
}
// Message User: kiểu dữ liệu chính
message User {
int32 id = 1;
string name = 2;
string email = 3;
int32 age = 4;
Role role = 5;
repeated string hobbies = 6; // Danh sách sở thích
}
// Request và Response messages
message GetUserRequest { int32 id = 1; }
message GetUserResponse { User user = 1; }
message CreateUserRequest {
string name = 1;
string email = 2;
int32 age = 3;
}
// Service với 2 RPC methods
service UserService {
rpc GetUser (GetUserRequest) returns (GetUserResponse);
rpc CreateUser (CreateUserRequest) returns (User);
}

5.4 Sử dụng generated code trong Go

Sau khi generate (xem Chương 6), bạn dùng code như sau:

chapter05/use_proto/main.go
package main
import (
"fmt"
pb "grpc-prereqs/gen/user" // Import generated code
"google.golang.org/protobuf/proto"
)
func main() {
// Tạo một User message – giống tạo struct Go bình thường
user := &pb.User{
Id: 1,
Name: "Nguyen Van An",
Email: "an@example.com",
Age: 22,
Role: pb.Role_ROLE_ADMIN,
Hobbies: []string{"coding", "reading", "coffee"},
}

// Serialize → bytes (đây là những gì gRPC gửi qua mạng)
data, err := proto.Marshal(user)
if err != nil { panic(err) }
fmt.Printf("Serialized: %d bytes\n", len(data))
// Deserialize ← bytes (đây là những gì gRPC nhận về)
var user2 pb.User
if err := proto.Unmarshal(data, &user2); err != nil { panic(err) }
fmt.Printf("Deserialized: Name=%s, Role=%s\n", user2.Name, user2.Role)
}

Tóm tắt chương

- File .proto là schema định nghĩa data types và service interface
- message = struct trong Go; repeated = slice; enum = iota constants
- Field number là định danh duy nhất — KHÔNG được thay đổi sau khi deploy
- Service định nghĩa 4 loại RPC: unary, server streaming, client streaming, bidirectional
- oneof đảm bảo chỉ một trong nhiều field được set tại một thời điểm

Câu hỏi / Bài tập tự luyện

- Viết file .proto cho một Product service với các fields: id, name, price, category, in_stock. Thêm enum cho Category.
- Thêm một RPC method SearchProducts(SearchRequest) returns (stream Product) vào service.
- Tại sao field number 1-15 encode nhỏ hơn field number 16-2047 trong Protobuf binary format?

Chương 6 — Code Generation Workflow

Mục tiêu: Thành thạo quy trình từ file .proto đến Go code sẵn sàng sử dụng.

6.1 Cài đặt công cụ

6.1.1 Cài protoc (Protocol Buffers Compiler)

protoc là trình biên dịch đọc file .proto và generate code.

Ubuntu 22.04
Cài protoc v25.x
sudo apt update
sudo apt install -y protobuf-compiler
Kiểm tra phiên bản
protoc --version
Output: libprotoc 25.x

macOS (dùng Homebrew)
brew install protobuf
protoc --version

Windows (PowerShell – chạy với Admin)
Dùng Chocolatey
choco install protoc
Hoặc tải trực tiếp từ GitHub:
https://github.com/protocolbuffers/protobuf/releases
Giải nén và thêm thư mục bin vào PATH

6.1.2 Cài Go plugins cho protoc

Tất cả hệ điều hành (trong Go project)
Plugin sinh Go message code
go install google.golang.org/protobuf/cmd/protoc-gen-go@v1.34
Plugin sinh gRPC service stubs
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@v1.3
Đảm bảo \$GOPATH/bin trong PATH
Ubuntu/macOS: thêm vào ~/.bashrc hoặc ~/.zshrc
export PATH="\$PATH:\$(go env GOPATH)/bin"
Windows: Thêm vào PATH qua System Properties → Environment Variables
%USERPROFILE%\go\bin

6.1.3 Cài gRPC package

terminal (trong thư mục project)
go get google.golang.org/grpc@v1.64
go get google.golang.org/protobuf@v1.34

6.2 Cấu trúc thư mục cho chapter 6

Cấu trúc thư mục	
chapter06/	
├── proto/	
│ └── user.proto	← file .proto bạn viết
├── gen/	
│ └── user/	
│ ├── user.pb.go	← generated: message types
│ └── user_grpc.pb.go	← generated: service stubs
└── main.go	← code dùng generated files

6.3 Chạy protoc để generate code

Ubuntu/macOS
Đứng ở thư mục chapter06
protoc \
--proto_path=proto \
--go_out=gen \
--go_opt=paths=source_relative \
--go-grpc_out=gen \
--go-grpc_opt=paths=source_relative \
proto/user.proto

Windows (PowerShell)
protoc `
--proto_path=proto `
--go_out=gen `
--go_opt=paths=source_relative `
--go-grpc_out=gen `
--go-grpc_opt=paths=source_relative `
proto/user.proto

6.4 Giải thích các flag của protoc

Flag	Ý nghĩa
--proto_path=proto	Thư mục tìm file .proto
--go_out=gen	Output thư mục cho file .pb.go (message types)
--go_opt=paths=source_relative	Giữ cấu trúc thư mục tương đối
--go-grpc_out=gen	Output thư mục cho file _grpc.pb.go (service)
--go-grpc_opt=paths=source_relative	Giữ cấu trúc thư mục tương đối

6.5 Kiểm tra output

terminal
Xem file được generate
ls -la gen/user/
user.pb.go ← Chứa struct User, Role enum, các methods

user grpc.pb.go ← Chứa UserServiceClient, UserServiceServer interfaces
Xem nhanh nội dung
head -50 gen/user/user.pb.go

6.6 Tự động hóa với Makefile

Thay vì gõ lệnh protoc dài mỗi lần, tạo Makefile để tiết kiệm thời gian:

Makefile
Makefile – chạy bằng: make proto
PROTO_DIR = proto
GEN_DIR = gen
proto:
protoc \
--proto_path=\$(PROTO_DIR) \
--go_out=\$(GEN_DIR) \
--go_opt=paths=source_relative \
--go-grpc_out=\$(GEN_DIR) \
--go-grpc_opt=paths=source_relative \
\$(PROTO_DIR)/*.proto
clean:
rm -rf \$(GEN_DIR)
.PHONY: proto clean

⚠ Commit generated code hay không?

Trong teamwork: commit file generated (.pb.go) vào git để mọi người không cần cài protoc. Đặt comment ở đầu file: 'DO NOT EDIT — generated by protoc'. Khi cập nhật .proto, regenerate và commit lại.

📌 Tóm tắt chương

- protoc là compiler đọc .proto và generate code cho nhiều ngôn ngữ
- protoc-gen-go và protoc-gen-go-grpc là 2 plugin Go cần thiết
- Sau khi generate: user.pb.go chứa structs, user_grpc.pb.go chứa service interfaces
- Dùng Makefile để tự động hóa việc generate code

🔗 Câu hỏi / Bài tập tự luyện

- Cài đặt protoc và cả hai plugin. Chạy protoc --version để kiểm tra.
- Generate code từ file user.proto của Chương 5. Mở file .pb.go và tìm struct User được generate.
- Thêm một field phone string = 7 vào User message, regenerate, và kiểm tra thay đổi trong .pb.go.

Chương 7 — Concurrency và Streaming trong ngữ cảnh gRPC

Mục tiêu: Liên kết goroutine/channel đã biết với streaming concept của gRPC để chuẩn bị tư duy trước khi viết gRPC streaming code.

7.1 4 loại RPC trong gRPC

Loại RPC	Client gửi	Server trả về	Ví dụ use case
Unary	1 request	1 response	GetUser(id) → User
Server Streaming	1 request	nhiều response	GetFeed() → stream Post
Client Streaming	nhiều request	1 response	stream File chunks → UploadID
Bidirectional	nhiều request	nhiều response	Chat: stream Msg ↔ stream Msg

7.2 Channel trong Go — Nền tảng tư duy Streaming

gRPC streaming hoạt động rất giống channel trong Go: một bên gửi dữ liệu liên tục, bên kia nhận và xử lý. Hãy nắm vững channel trước khi học streaming.

chapter07/channels/main.go
package main
import (
"fmt"
"time"
)
// Producer gửi dữ liệu vào channel – giống gRPC client streaming
func producer(ch chan<- int, n int) {
defer close(ch) // Đóng channel khi gửi xong – giống CloseSend() trong gRPC
for i := 1; i <= n; i++ {
ch <- i
fmt.Printf("Producer gửi: %d\n", i)
time.Sleep(100 * time.Millisecond)
}
}
// Consumer nhận dữ liệu từ channel – giống gRPC server nhận stream
func consumer(ch <-chan int) {
sum := 0
// range tự động dừng khi channel được close
for v := range ch {
sum += v
fmt.Printf("Consumer nhận: %d, tổng hiện tại: %d\n", v, sum)
}
fmt.Printf("Tổng cuối cùng: %d\n", sum)
}
func main() {
ch := make(chan int, 5) // Buffered channel, buffer size 5
go producer(ch, 5)
consumer(ch) // Chạy trên main goroutine
}

7.3 Mô phỏng Server Streaming với Channel

<code>chapter07/server_stream_sim/main.go</code>
<code>package main</code>
<code>import (</code>
<code> "fmt"</code>
<code> "time"</code>
<code>)</code>
<code>// StreamResult mô phỏng dữ liệu gRPC server gửi về</code>
<code>type StreamResult struct {</code>
<code> Data string</code>
<code> Error error</code>
<code>}</code>
<code>// serverStreamSimulator mô phỏng server streaming trong gRPC</code>
<code>// Trong gRPC thật: server gọi stream.Send() để gửi từng item</code>
<code>func serverStreamSimulator(query string) <-chan StreamResult {</code>
<code> ch := make(chan StreamResult)</code>
<code> go func() {</code>
<code> defer close(ch)</code>
<code> // Giả lập server tìm kiếm và trả về từng kết quả</code>
<code> items := []string{"Kết quả 1", "Kết quả 2", "Kết quả 3"}</code>
<code> for _, item := range items {</code>
<code> time.Sleep(200 * time.Millisecond) // Giả lập delay xử lý</code>
<code> ch <- StreamResult{Data: fmt.Sprintf("[%s] %s", query, item)}</code>
<code> }</code>
<code> }()</code>
<code> return ch</code>
<code>}</code>
<code>func main() {</code>
<code> fmt.Println("Bắt đầu nhận stream từ server...")</code>
<code></code>
<code> // Client nhận từng item ngay khi server gửi</code>
<code> // Trong gRPC thật: client gọi stream.Recv() trong vòng lặp</code>
<code> for result := range serverStreamSimulator("Go gRPC") {</code>
<code> if result.Error != nil {</code>
<code> fmt.Println("Lỗi:", result.Error)</code>
<code> break</code>
<code> }</code>
<code> fmt.Println("Nhận:", result.Data)</code>
<code> }</code>
<code> fmt.Println("Stream kết thúc.")</code>
<code>}</code>

7.4 Goroutine và Context — Quản lý vòng đời

Trong gRPC streaming, bạn cần quản lý vòng đời (lifecycle) của stream: khi nào bắt đầu, khi nào kết thúc, khi nào hủy. Go's context.Context là công cụ cho việc này.

<code>chapter07/context_demo/main.go</code>
<code>package main</code>
<code>import (</code>
<code> "context"</code>
<code> "fmt"</code>
<code> "time"</code>
<code>)</code>
<code></code>
<code>// streamWithContext mô phỏng streaming có timeout</code>

```
// Trong gRPC: mọi stream đều nhận ctx context.Context
func streamWithContext(ctx context.Context, ch chan<- int) {
    defer close(ch)
    for i := 1; ; i++ {
        select {
        case <-ctx.Done():
            // Context bị hủy (timeout hoặc cancel)
            fmt.Println("Stream bị hủy:", ctx.Err())
            return
        case ch <- i:
            fmt.Printf("Gửi: %d\n", i)
            time.Sleep(300 * time.Millisecond)
        }
    }
}

func main() {
    // Tạo context với timeout 1 giây
    // Nếu stream chạy quá 1s, tự động bị hủy
    ctx, cancel := context.WithTimeout(context.Background(), 1*time.Second)
    defer cancel()

    ch := make(chan int, 1)
    go streamWithContext(ctx, ch)

    for v := range ch {
        fmt.Printf("Nhận: %d\n", v)
    }
}
```

💡 Liên hệ với gRPC Streaming

Khi học gRPC streaming, bạn sẽ thấy: `stream.Send(msg) ≈ ch <- msg | stream.Recv() ≈ v := <- ch | io.EOF ≈ close(ch) | ctx.Done() = client hủy hoặc timeout`. Tư duy channel → tư duy streaming.

🔗 Tóm tắt chương

- gRPC có 4 loại RPC: unary, server streaming, client streaming, bidirectional
- Channel trong Go là analog tốt nhất để hiểu streaming concept
- producer/consumer pattern với channel ≈ client/server streaming trong gRPC
- context.Context quản lý timeout và cancellation của stream

🔧 Câu hỏi / Bài tập tự luyện

19. Sửa `serverStreamSimulator` để nó nhận context và tự dừng khi context bị cancel.
20. Viết mô phỏng Bidirectional streaming: cả client và server đều gửi/nhận trên 2 channel riêng.
21. Trong gRPC thật, `stream.Recv()` trả về `io.EOF` khi stream kết thúc. Điều tương đương trong channel là gì?

Chương 8 — Thiết kế cấu trúc thư mục Project

Mục tiêu: Hiểu cách tổ chức code Go project theo chuẩn, dễ mở rộng — và tổ chức lại toàn bộ code từ Chương 1-7 vào một project thống nhất.

8.1 Cấu trúc thư mục đề xuất

grpc-prereqs/ - Toàn bộ project	
grpc-prereqs/	
├── go.mod	← Go module declaration
├── go.sum	← Checksums của dependencies
├── Makefile	← Automation scripts
├── README.md	← Hướng dẫn project
├── proto/	← TẤT CẢ file .proto
│ ├── user/	
│ │ └── user.proto	
│ └── product/	
│ └── product.proto	
├── gen/	← Code được generate (KHÔNG sửa tay)
│ ├── user/	
│ │ ├── user.pb.go	
│ │ └── user_grpc.pb.go	
│ └── product/	
│ ├── product.pb.go	
│ └── product_grpc.pb.go	
├── chapter01/	← Code thực hành Chương 1
│ ├── server/	
│ │ └── main.go	
│ └── client/	
│ └── main.go	
├── chapter02/	
│ └── http_compare/	
│ └── main.go	
├── chapter03/	
│ └── manual_rpc/	
│ └── main.go	
├── chapter04/	
│ └── serialization/	
│ └── main.go	
├── chapter05/	
│ └── use_proto/	
│ └── main.go	
├── chapter06/	
│ └── (cùng cấp với gen/)	
└── chapter07/	
│ ├── channels/	
│ │ └── main.go	
│ ├── server_stream_sim/	
│ │ └── main.go	
│ └── context_demo/	
│ └── main.go	

8.2 Giải thích vai trò từng thư mục

Thư mục	Vai trò	Ai sửa?
proto/	Source of truth — định nghĩa API contract	Developer
gen/	Code tự động generate từ proto/	protoc (KHÔNG sửa tay)
chapter0x/	Code thực hành và học tập	Developer
go.mod	Khai báo module name và Go version	Developer / go get
Makefile	Scripts tự động hóa (proto generate, test, build)	Developer

8.3 go.mod — Module declaration

go.mod
module grpc-prereqs
go 1.22
require (
google.golang.org/grpc v1.64.0
google.golang.org/protobuf v1.34.0
golang.org/x/net v0.24.0
)

8.4 Makefile hoàn chỉnh

Makefile
Makefile cho grpc-prereqs project
PROTO_DIR = proto
GEN_DIR = gen
Generate tất cả file proto
proto:
@echo "Generating proto files..."
for f in \$(shell find \$(PROTO_DIR) -name "*.proto"); do \
dir=\$(dirname \$\$f sed "s \$(PROTO_DIR)/ "); \
protoc --proto_path=\$(PROTO_DIR) \
--go_out=\$(GEN_DIR) \
--go_opt=paths=source_relative \
--go-grpc_out=\$(GEN_DIR) \
--go-grpc_opt=paths=source_relative \
\$\$f; \
done
@echo "Done!"
Chạy tất cả chapter examples
run-ch01-server:
go run chapter01/server/main.go
run-ch01-client:
go run chapter01/client/main.go
Build tất cả binaries
build:
go build ./...
Chạy tests

test:
go test ./...
Xóa generated files
clean-gen:
rm -rf \$(GEN_DIR)
.PHONY: proto build test clean-gen

8.5 .gitignore gợi ý

.gitignore
Build outputs
bin/
*.exe
Không ignore gen/ - commit generated code
(giúp teammates không cần cài protoc)
Test artifacts
coverage.out
IDE
.idea/
.vscode/
*.swp

Tóm tắt chương

- Tách proto/ (source) và gen/ (generated) rõ ràng — không sửa file trong gen/
- Mỗi chapter/feature có thư mục riêng để code dễ tìm
- go.mod khai báo module name — phải khớp với import paths trong code
- Makefile tự động hóa các task lặp lại như generate proto, build, test
- Commit gen/ vào git để teammates không cần cài protoc

Chương 9 — Lỗi thường gặp và cách xử lý

Tổng hợp các lỗi phổ biến từ tất cả các chương, kèm nguyên nhân gốc rễ và hướng xử lý cụ thể.

9.1 Lỗi cài đặt môi trường

Lỗi	Nguyên nhân	Cách xử lý
protoc: command not found	protoc chưa được cài hoặc không có trong PATH	Cài lại protoc. Thêm /usr/local/bin (Linux) hoặc %USERPROFILE%\go\bin (Windows) vào PATH. Chạy source ~/.bashrc
protoc-gen-go: program not found	Plugin protoc-gen-go chưa được cài hoặc GOPATH/bin chưa trong PATH	Chạy: go install google.golang.org/protobuf/cmd/protoc-gen-go@v1.34. Thêm \$(go env GOPATH)/bin vào PATH
plugin "protoc-gen-go-grpc" not found	Plugin grpc chưa cài	Chạy: go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@v1.3

9.2 Lỗi khi viết file .proto

Lỗi	Nguyên nhân	Cách xử lý
Field number already used	Hai fields trong cùng message dùng chung field number	Mỗi field trong message phải có field number duy nhất. Kiểm tra và đổi field number bị trùng
The first enum value must be zero	Giá trị đầu tiên trong enum không phải 0 (proto3 bắt buộc)	Thêm ENUM_UNSPECIFIED = 0 làm giá trị đầu tiên của mọi enum
option go_package not set	Thiếu option go_package trong file .proto	Thêm: option go_package = "module_name/gen/package_name"; vào đầu file .proto

9.3 Lỗi kết nối TCP/Network

Lỗi	Nguyên nhân	Cách xử lý
bind: address already in use	Port đang được dùng bởi process khác	Ubuntu/macOS: lsof -i :PORT để tìm process, kill -9 PID để dừng. Windows: netstat -ano findstr PORT
connection refused	Server chưa chạy hoặc sai địa chỉ/port	Kiểm tra server đã chạy chưa. Đảm bảo địa chỉ host:port khớp giữa server và client
i/o timeout	Server không phản hồi trong thời gian cho phép	Kiểm tra logic server có block không (deadlock?). Tăng timeout hoặc thêm context.WithTimeout

9.4 Lỗi Serialization

Lỗi	Nguyên nhân	Cách xử lý
-----	-------------	------------

proto: (line N): unknown field "fieldname"	Tên field trong JSON không khớp với proto (khác case)	Protobuf dùng camelCase trong JSON. VD: my_field trong proto → myField trong JSON
cannot parse invalid wire-format data	Dữ liệu nhận về không phải Protobuf hợp lệ (có thể là JSON hoặc binary lẫn lộn)	Đảm bảo cả 2 bên dùng proto.Marshal/Unmarshal, không trộn JSON và Protobuf

9.5 Lỗi Concurrency

Lỗi	Nguyên nhân	Cách xử lý
fatal error: concurrent map read and write	Nhiều goroutine truy cập map cùng lúc không có mutex	Dùng sync.Mutex hoặc sync.RWMutex bảo vệ map. Hoặc dùng sync.Map
goroutine leak (memory tăng liên tục)	Goroutine bị stuck chờ channel mà không bao giờ được gửi/đóng	Luôn đảm bảo channel được close hoặc dùng context để cancel goroutine. Dùng goleak để detect
deadlock: all goroutines are asleep	Tất cả goroutine đều đang chờ nhau — không ai tiến được	Kiểm tra channel send/receive có cân bằng không. Buffered channel có thể giúp. Thêm select với default

9.6 Lỗi Import và Module

Lỗi	Nguyên nhân	Cách xử lý
cannot find module providing package	Package chưa được khai báo trong go.mod hoặc chưa download	Chạy: go get <package>@version. Sau đó go mod tidy để dọn dẹp
imported and not used	Import package nhưng không dùng đến trong code	Xóa import không dùng. Nếu cần side-effect import: dùng _ "package/name"

Chương 10 — Tổng kết

10.1 Bảng ôn tập khái niệm toàn tài liệu

Khái niệm	Định nghĩa ngắn gọn	Chương
TCP	Giao thức truyền dữ liệu đáng tin cậy qua mạng	1
Port	Số cổng định danh một dịch vụ trên máy tính	1
Client-Server	Client chủ động kết nối; Server lắng nghe	1
HTTP/1.1	Giao thức web text-based, tuần tự	2
HTTP/2	Binary, multiplexing, compression — nền tảng gRPC	2
Multiplexing	Nhiều stream song song trên 1 TCP connection	2
RPC	Gọi hàm trên máy khác như gọi local	3
REST	API thiết kế theo resource + HTTP method	3
Serialization	Chuyển struct thành bytes để gửi qua mạng	4
Protobuf	Binary serialization format nhỏ, nhanh, đa ngôn ngữ	4,5
message	Kiểu dữ liệu trong .proto — tương đương struct Go	5
service	Định nghĩa các RPC methods trong .proto	5
field number	Định danh duy nhất của field trong binary format	5
protoc	Compiler chuyển .proto thành code Go/Java/Python...	6
Goroutine	Lightweight thread của Go — nền tảng concurrency	7
Channel	Cơ chế giao tiếp an toàn giữa goroutines	7
context.Context	Quản lý timeout và cancellation	7
Streaming RPC	Gửi/nhận nhiều message trong một RPC call	7

10.2 Lộ trình học gRPC tiếp theo

Bạn đã có đủ nền tảng. Đây là lộ trình học gRPC được gợi ý:

Giai đoạn	Nội dung	Thời gian gợi ý
Bước 1	Viết gRPC server + client Unary đầu tiên	2-3 ngày
Bước 2	Implement cả 4 loại RPC (unary, server/client/bi streaming)	1 tuần
Bước 3	Error handling, status codes, metadata trong gRPC	3-4 ngày
Bước 4	Interceptors (middleware) — auth, logging, retry	1 tuần
Bước 5	gRPC với TLS/mTLS (bảo mật kết nối)	3-4 ngày
Bước 6	Reflection, grpcurl để debug gRPC services	2 ngày
Bước 7	gRPC-Gateway: expose gRPC as REST API	1 tuần

Bước 8	Production: load balancing, health checks, tracing	2 tuần
--------	--	--------

10.3 Tài nguyên học tập

Documentation chính thức

- grpc.io — Tài liệu chính thức của gRPC (có ví dụ Go)
- protobuf.dev — Language Guide và API Reference của Protobuf
- pkg.go.dev/google.golang.org/grpc — Go API Reference

Sách gợi ý

- "gRPC: Up and Running" — Kasun Indrasiri, Danesh Kuruppu (O'Reilly)
- "Production Kubernetes" — Josh Rosso (bao gồm gRPC trong microservices)

Repo mẫu để tham khảo

- github.com/grpc/grpc-go/examples — Official Go gRPC examples
- github.com/grpc-ecosystem/go-grpc-middleware — Middleware collection
- github.com/grpc-ecosystem/grpc-gateway — gRPC-REST Gateway

Công cụ hữu ích

- [grpcurl](#) — curl cho gRPC (debug từ terminal)
- [grpcui](#) — GUI web để test gRPC services
- [evans](#) — Interactive gRPC client
- [buf](#) — Modern Protobuf toolchain (thay thế protoc phức tạp)

Lời khuyên cuối cùng

Đừng chỉ đọc — hãy code. Mỗi khái niệm trong tài liệu này trở nên rõ ràng hơn gấp 10 lần khi bạn tự tay gõ và chạy code. Khi gặp lỗi, đừng nản — lỗi là bằng chứng bạn đang học đúng hướng. Chúc bạn học tốt và sớm build được gRPC services thực tế!